

M O U S S E

Mousse — Foundation, Subagent & Worktree Review

Architecture and code-quality review against the Mousse vision

29 September 2026

Prepared for the product owner

Contents

1. Executive Summary	3
2. The vision and what it requires	4
Method and legend	4
3. Part 1: Foundation review	5
3.1 Fatal flaws: block the vision	5
3.2 Security issues to fix now	6
3.3 Major structural issues	8
3.4 Code quality and hygiene	9
3.5 Strengths to keep	9
3.6 Target foundation: six primitives	9
3.7 Suggested order	10
4. Part 2: Subagent and worktree review	11
4.1 Today's turn and subagent flow	11
4.2 Critical findings	12
4.3 Major findings: subagents	13
4.4 Major findings: worktrees and git	13
4.5 Minor findings and code quality	14
4.6 Strengths to keep	14
4.7 Recommended re-architecture	14
5. Roadmap	16

1. Executive Summary

Verdict. As a single-owner, single-machine agent orchestrator, Mousse is well built. Crash recovery, durable queues, the workflow engine, process-tree ownership and lifecycle draining are more careful than most projects of this size. The git and worktree layer is the best-engineered part.

The vision, however, needs five things the foundation does not have yet: **identity, authorization, durable ordered event streams, a transport-independent client, and safe concurrency on a shared conversation.** They should be built into the core before rooms, bots or Mousse Bridge, or each feature will grow its own ad-hoc permission, sync and identity logic. The subagent system is the weakest area: it works on the happy path but breaks on restart and on clean-but-silent exits.

Top 10 things to fix

1. **Two turns can run on one thread at once** (F4). Add a per-thread admission mutex now.
2. **Daemon restart leaves a parent agent waiting forever** (C1). Persist batches and wakes; reconcile on startup.
3. **Retry after a merge conflict can commit conflict markers** (C3). Guard checkpoints against MERGE_HEAD / REVERT_HEAD.
4. **Clean exit with no report hangs a headless agent** (C2). Treat exit 0 without completion as needs_review.
5. **Webhook channel lets any web page trigger an agent turn** (S1).
6. **Tools give any sender effectively a host shell** (F2). Add one authorization choke point and realpath containment.
7. **Worker reports are injected as user instructions** (C4). Wrap them in an author-tagged block.
8. **Event sync is broken for remote devices** (F3): no chat events, subscribe rejected, cursor bug.
9. **Corrupt config silently replaced with defaults** (S4); owner lease races (S5); no crash handler (S9).
10. **No identity model** (F1): messages have a role but no author. Build Principal + event log as the kernel.

Scorecard

Area	Rating	Reason
Single-owner orchestration	Strong	Crash recovery, durable queues, lifecycle draining, process-tree ownership.
Workflow engine and run store	Strong	Fsynced NDJSON journal, deterministic run IDs, in-doubt effect recovery.
Git and worktree layer	Strong	One lock order, write-ahead journal, undo by compensating commit. Edge cases only.
Identity	Blocker for vision	Messages carry only a role; no author, sequence or parent.
Authorization and tool authority	Blocker for vision	Owner token grants everything; main-turn tools allow all.
Event log and sync	Blocker for vision	Mutable JSON transcript, in-memory rings, three event systems.
Concurrency on shared threads	Blocker for vision	Admission window lets two turns run at once.
Client and transport	Blocker for vision	Clients tied to a local socket and to <code>window.mousse</code> .
Subagent system	Needs work	Text-parsed commands, in-memory state, breaks on restart.
Channels	Needs work	No durable inbox/outbox, no dedupe, no mention gating.
Code structure and hygiene	Needs work	God classes, three UI copies, no ESLint, Ubuntu-only CI.
Security posture	Needs work	Nine issues listed; several are quick to fix.

2. The vision and what it requires

Mousse should be a single agentic application where the owner manages workflows, remotes into various devices and runs chats from different devices over "Mousse Bridge" (not yet built), and supports team chats where people create rooms, talk to each other, and involve bots that can create threads, reply, and be talked to. That requires:

- **Identity:** who said or did what.
- **Authorization:** what they may do, on which resource.
- **Durable, ordered event streams:** every client can resume from a cursor.
- **A transport-independent client:** the same client works locally and over Bridge.
- **Safe concurrency on shared conversations:** one writer per thread at a time.
- **Clean service boundaries:** replacing god classes and process-global state.

Method and legend

Part 1 used seven parallel reviews (daemon core, protocol and remote control, orchestrator, data and persistence, channels and workflows, renderer UI, tests and tooling). Part 2 used three parallel reviews (subagent lifecycle, worktree/git layer, child merge, undo and publish) plus direct code reading. Fatal and security-critical claims were re-checked in code. Typecheck is clean on both tsconfig projects. **Tests were not run.**

Label	Meaning
CRITICAL	Critical / fatal: blocks the vision or can corrupt data or hang work.
SECURITY	Security: exposes access, secrets or untrusted input paths.
MAJOR	Major: significant structural or reliability issue.
MINOR	Minor: hygiene, dead code, duplication, test gaps.
Verified	Verified in code (shown as a check mark in the original review notes).
Unverified	Likely, but not fully verified (warning marker in the original notes). "Detail unverified" means the core claim is verified and a detail is not.

3. Part 1: Foundation review

3.1 Fatal flaws: block the vision

F1 No identity anywhere

CRITICAL

Verified

What's wrong. ChatMessage carries only a role (user, assistant, system): no author, sequence, reply parent or revision. Send requests, queue items, execution actors, workflow manifests and approvals have no human principal. Channels receive a userId and then drop it; agent reports are injected as hidden user messages.

Why it matters. The model cannot tell a bot from the owner, and rooms cannot attribute who said or approved anything. The UI has 37 hard-coded role checks.

Fix direction. Introduce a Principal and give every message authorId, parentId, revision and kind.

```
src/shared/types.ts:415-487 (ChatMessage)
registerRunMethods.ts:89 (approval attributed to connection id)
ChannelRouter.ts:337 (userId dropped)
```

F2 Anyone who can send a message effectively gets a shell on the host

CRITICAL

Verified

What's wrong. One owner token grants everything locally (PTYs, files, git, settings, device pairing). Paired remote devices get five device-wide scopes (all by default, including terminal) through the same dispatcher with no profile binding. The main turn passes `toolAccess { allows: () => true }`; bash is a raw shell; read/grep/ls/find accept any absolute path; write/edit check paths by text only. Child CLI agents always run with permission checks bypassed and inherit the daemon environment. Allowlisted Discord/Telegram users can list and bind any thread and run `/model --global`.

Why it matters. Profiles separate data but are not a security boundary. Any future room member or bot would inherit host-level power.

Fix direction. One authorize(principal, action, resource) choke point used by handlers, remote dispatcher, channel ingress and every tool; generalize ExecutionPolicyService; realpath containment for all file tools; scrubbed child env; capability grants for bots.

```
server.ts:705 (owner token)
MmsProfileServices.ts:271-283 (remote scopes)
OrchestratorService.ts:2849 (allows: () => true)
agentPermissionFlags.ts (dangerous flags)
```

F3 No durable event log; sync is fragmented

CRITICAL

Verified

What's wrong. The transcript is a mutable JSON array rewritten whole on every persist (after each tool call, every 500 ms while streaming), kept in three semi-canonical copies; the loader picks by modification time. Events live in 512-entry in-memory rings that restart at 0. MmsEventBus declares thread-message events that nothing emits, so remote devices never receive chat messages. Remote subscribe is rejected before replay code runs. hello_ok reports the global counter while clients get per-profile sequence numbers, so the client drops events below its seeded cursor (masked because the GUI always resnapshots).

Why it matters. Remote devices, rooms and resume-after-disconnect all depend on ordered, replayable events.

Fix direction. Append-only event log with server-assigned seq per stream; one event system; subscribe(streamId, afterSeq). Fix the cursor bug immediately.

```
events.ts:12-16 (declared, never emitted)
remoteDispatcher.ts:274 (subscribe rejected), :321 (replay)
client.ts:268-275, 490-494 (cursor seed)
messages.json / conversation-state.json / llm-context.json
```

F4 Two turns can run on the same thread at once

CRITICAL

Verified

What's wrong. Send A passes `isTurnRunning()` (which only checks `activeTurn`), acquires and stores a lease, then awaits recovery. `activeTurn` is not set until later. Send B in that window (another client, channel or queue drain) passes the external-lease check, which excludes the session's own token, and reuses the lease. Both run, and A's finally releases the lease B is using. A second window exists after `activeTurn` is cleared. Tests only cover different threads.

Why it matters. Produces a corrupted transcript and model context; becomes common once several people share a room.

Fix direction. Per-thread admission mutex now; later a per-stream serial executor that owns admission through lease release.

```
OrchestratorService.ts:2537 (lease stored), :2551 (await recovery), :2767 (activeTurn set), :2319 (own token excluded), :2509 (lease reused), :2952 (activeTurn = null)
```

F5 Clients are welded to the local machine

CRITICAL

Verified

What's wrong. The protocol requires a local `net.Socket` and a token read from disk; the GUI hard-codes `profiles.bind {profile:'default'}`. The renderer calls `window.mouse.*` from about 315 places in 38 files with no client abstraction. Electron main runs file and git operations itself, trusting a renderer-supplied `cwd`. The orchestrator keeps process-global "selected thread" state.

Why it matters. Mousse Bridge and any web or mobile client cannot reuse the current client code.

Fix direction. A typed `MousseClient` SDK (request + subscribe with cursor); Electron main stops doing file/git work; remove process-global selection.

```
registerGuiIpc.ts:1650-1720 (main-process file/git)
boundSession, worktrees.setRepoRoot (global state)
```

3.2 Security issues to fix now

S1 Webhook lets any web page trigger an agent turn

SECURITY

Verified

What's wrong. Off by default, but when enabled: `allowAllUsers` is true, the secret is empty, the body is parsed as JSON regardless of content-type (CSRF from any page), with no size cap and no Origin check. The caller chooses `userId`, and callers sharing a `chatId` can receive each other's replies.

Why it matters. Any web page the owner visits could start an agent turn.

Fix direction. Require a secret, enforce content-type, size cap and Origin check, and bind identity server-side.

```
ChannelStore.ts:23-29
WebhookAdapter.ts:135-182
```

S2 A renderer XSS would give full owner access

SECURITY

Verified

Detail unverified

What's wrong. The main window has `sandbox:false` and `webViewTag:true`, there is no CSP, `openExternal` accepts any scheme, the renderer can approve device pairing with arbitrary scopes, and IPC handlers do not check the sender.

Why it matters. Any script injection in the renderer becomes full control of the host.

Fix direction. Enable sandbox, add a CSP, restrict URL schemes, validate IPC senders, and require out-of-band pairing approval.

```
main/index.ts:220-223
registerGuiIpc.ts:1500, 2034
```

S3 Idempotency hash drops nested keys

SECURITY

Verified

What's wrong. The array replacer in `JSON.stringify` filters keys at every depth, so different `settings.set` payloads hash the same. The store is in memory, and concurrent duplicates run twice.

Why it matters. A different request can be treated as a duplicate and silently skipped.

Fix direction. Canonical hashing over the full payload; persist the store; reserve keys atomically.

```
control/storage/idempotencyStore.ts:40
```

S4 Corrupt `mongoose.conf` silently replaced with defaults

SECURITY

Verified

What's wrong. Reading a corrupt config yields defaults, which wipes channels, schedules and providers. `projects.json` and `secret refs` behave the same.

Why it matters. One bad write can erase configuration without warning.

Fix direction. Fail loudly, keep a backup, and never overwrite on parse error.

```
MousseConfigStore.ts:246-256
```

S5 Owner lease can end up with two daemons or none

SECURITY

Verified

What's wrong. Stale reclaim re-reads then unlinks without a lock. The heartbeat is never called in production, so a reused PID can wedge the lease.

Why it matters. Two daemons can run against one profile, or none can start.

Fix direction. Lock around reclaim; wire the heartbeat; validate liveness beyond PID.

```
MmsOwnerLease.ts:185-188
```

S6 Secrets at rest are inconsistent

SECURITY

Unverified

What's wrong. MCP OAuth tokens are plaintext, non-atomic, default permissions. The control-store key is derived from `user@host:dir` (obfuscation). Device private keys are plaintext; headless Node hosts store provider keys in plaintext.

Why it matters. A local file read yields tokens and keys.

Fix direction. Use OS credential storage or a real key, atomic writes and restricted permissions.

```
McpOAuthProvider.ts:52-72  
controlStore.ts:157
```

S7 Browser automation can reach the daemon's own loopback services

SECURITY

Unverified

What's wrong. Browser validation may allow loopback targets such as the webhook port (18789) and OAuth callback (8791).

Why it matters. A browsing agent could call internal services.

Fix direction. Block loopback and private ranges for automated browsing.

```
shared/browser/validation.ts:31-35
```

S8 Windows quoting may be escapable

SECURITY

Unverified

What's wrong. shellQuote only escapes ASCII single quotes, while PowerShell also treats U+2018 and U+2019 as quotes. It is used on prompts passed to CLI agents with approvals off.

Why it matters. Prompt text could break out of quoting.

Fix direction. Avoid shell quoting: pass arguments as arrays or via stdin.

```
headlessCommand.ts:33-37
```

S9 No unhandledRejection / uncaughtException handler, no file log

SECURITY

Verified

What's wrong. No global handlers exist and there is no file log.

Why it matters. One stray rejection kills every running agent without a trace.

Fix direction. Add handlers, a rotating file log, and crash-safe shutdown.

3.3 Major structural issues

Topic	Observation
God classes	OrchestratorService.ts 5,487 lines; LlmClient.ts 2,880; WorkflowRunService.ts 2,866; registerGuilpc.ts 2,111; protocol/handlers.ts 1,932 (163-case switch; <code>isObject(p) ? p : {}</code> 132 times); SettingsPage.tsx 1,995; ThreadDataStore.ts 1,355.
Six turn engines	Main, channel, schedule, workflow chat, runAgentDefinition, MousseAgentService. Budgets only for trusted agents; the main loop is unbounded. Plain scheduled prompts run with no thread, lease or persistence.
Composition	MmsProfileServices constructs about 35 services positionally with late setters. MousseMainService extends MmsProfileServices (installation IS the default profile). ProfileHost opens a second ProfileManager and config store over the same file.
Transport in domain	Protocol server event wiring sets thread activity state (server.ts:374-405). Every threads.updated pushes the entire thread list.
Protocol versioning	Hard <code>protocolVersion !== 1</code> check. Three dialects: local framing; Control 2.0 with aliased fields; an unimplemented OpenAPI/HTTP spec. docs/WIRE_PROTOCOL.md is referenced in 7 files but missing. Noise handshake may omit MixKey(e) in psk mode (unverified); its test vectors are self-generated.
Channels	No durable inbox/outbox (in-memory promise chain; Telegram offset in memory), no dedupe, failed replies dropped, no mention gating (every group message becomes a prompt), adapters are a closed hard-coded switch.
"Agent" naming	Means four things: runtime instance, versioned spec, config materializer, workflow pinning adapter.
Blocking I/O	492 sync fs calls in src/mms plus execFileSync git on request paths (up to 30 s).
Frontend	One ~60-field Zustand store mixes server cache and UI state; messages ordered by timestamp; optimistic sends reconciled by content equality; thread list polled; no virtualization; no error boundaries; GUI gives up after 12 reconnect attempts.

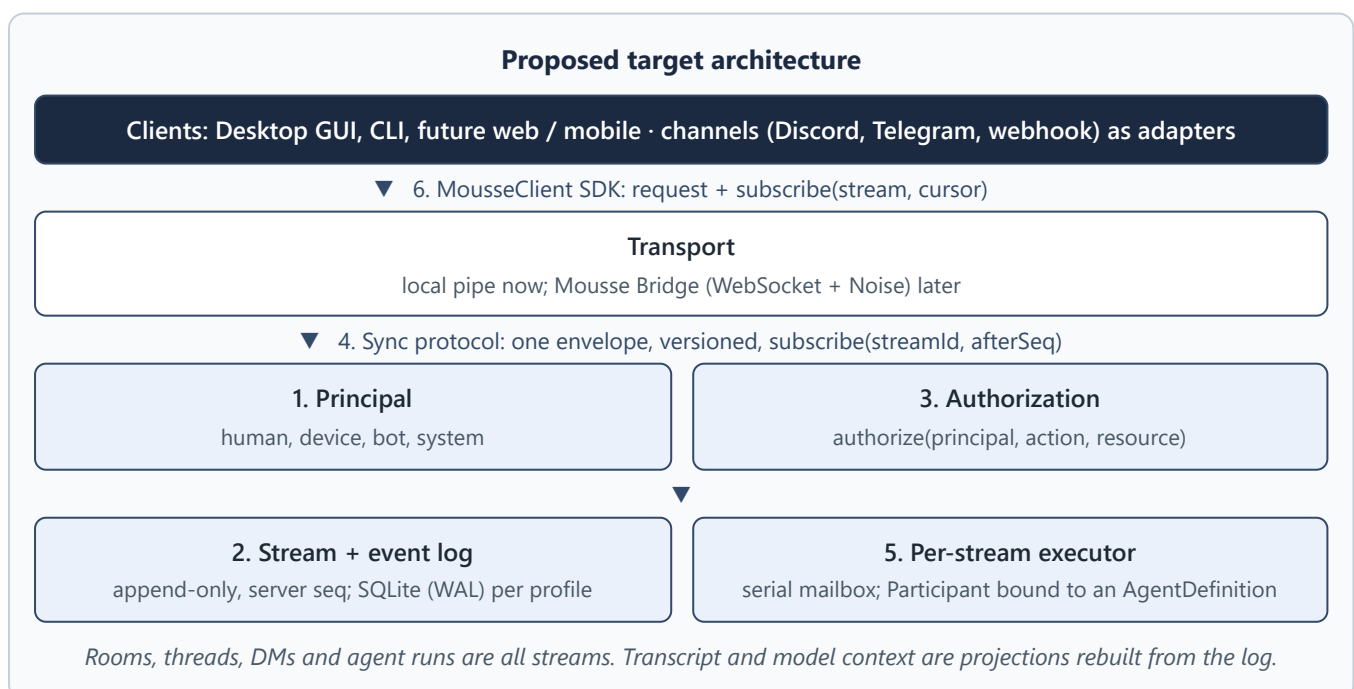
3.4 Code quality and hygiene

- **Verified** Three copies of the chat UI: `src/components/agent-elements` (41 files, dead, outside tsconfig), `src/renderer/components/agent-elements` (41, only file-attachment used), and the live `src/renderer/chat/components/agent-elements` (45). Copies have diverged.
- No ESLint/Prettier (yet 3 eslint-disable comments); 74 swallowed catches; no floating-promise checks.
- Timing hacks: `setTimeout` “wait 30 ms so UI paints”; a 5x2 s poll for an in-flight git commit.
- CI is Ubuntu-only although Windows is the primary target.
- Renderer tests run in a Node environment; there are no component-render tests.
- Unused deps: `openai`, `@jmondi/oauth2-server`, `motion`; `ai` is types-only but a runtime dependency. Three icon libraries and about four markdown/highlighting stacks.
- Docs: 158 files, 134 implementation plans, no “current” marker; `ARCHITECTURE.md` says “local-only, no remote” while about 1,100 lines of remote code ship.
- **Positives:** typecheck clean, 303 test files, strong daemon coverage, CI actions pinned by SHA, `any` is rare (106 uses in 818 files).

3.5 Strengths to keep

- **AtomicFs** (fsync of file and directory).
- **Workflow engine and run store:** fsynced NDJSON journal, deterministic run IDs, recovery of in-doubt effects, pinned agent snapshots. A template for event streams.
- **ExecutionPolicyService, TaskWriterAuthority, assertEpisodePath.**
- **DomainHandlerRegistry:** per-method validation, sealed, capability-gated.
- **Durable queue** claim/accept provenance and OwnedWorkBarrier.
- **Process-tree ownership** and sequenced PTY replay.
- **Browser worker isolation** with scrubbed env.
- **Renderer** `mousseToUI` adapter and the `platformRequest` client pattern.

3.6 Target foundation: six primitives



#	Primitive	What it means
1	Principal	Human, device, bot or system; server-resolved from each connection or channel identity. The owner is a principal with an admin role.
2	Stream + event log	Append-only log with server-assigned seq. Messages carry authorId, parentId, revision, kind. Recommend SQLite (WAL) per profile. This disagrees with the collaboration plan's separate room store, which would create two message models.
3	One authorization choke point	authorize(principal, action, resource) used by protocol handlers, remote dispatcher, channel ingress and every tool dispatch. Generalize ExecutionPolicyService; realpath containment for all file tools; scrubbed child env; bot capability grants scoped to rooms/projects.
4	Sync protocol	subscribe(streamId, afterSeq), authorized and filtered per principal; one envelope with version and feature negotiation; one event system; pluggable transport.
5	Per-stream executor	Serial mailbox per thread owns admission through lease release (fixes F4). One TurnRuntime/Participant interface; a bot is a Participant bound to an AgentDefinition; execution needs explicit invocation (mention or command).
6	Typed MousseClient SDK	request + subscribe(stream, cursor); IPC implementation now, Bridge later; used by renderer, CLI, future web/mobile. Host-local features are optional capabilities; Electron main stops doing file/git work.

3.7 Suggested order

Phase	Work
1. Now (days)	Fix S1 to S5 and S9; fix F4 with a per-thread admission mutex; fix the event cursor bug; delete dead UI copies; add ESLint (no-floating-promises) and a Windows CI job.
2. Kernel	Principal, stream/event log in SQLite, per-stream executor; migrate threads onto streams.
3. Authorization + client SDK	Migrate handlers.ts into DomainHandlerRegistry; split OrchestratorService.
4. Mousse Bridge	Transport under the unified protocol, principal-bound device credentials, cursor resume, Noise checked against external vectors.
5. Rooms + bots	Mostly product work on the kernel; channels become adapters with durable inbox/outbox.

Note on the collaboration plan. [docs/mousse-collaboration-plan.md](#) is good on product rules (explicit invocation, attributed requests, isolated worktrees) but misses F3 to F5, the tool-authority gap and the Electron exposure, and underestimates the storage and event-layer change.

4. Part 2: Subagent and worktree review

Method: three parallel reviews (subagent lifecycle, worktree/git layer, child merge + undo + publish) plus direct code reading. Tests were not run.

Git layer: the best-engineered part of Mousse. One lock order (thread lease then repository lease), write-ahead journaling with pinned SHAs, undo by compensating commit (no history rewrite), conservative GC, git args always arrays. Problems are edge cases and state-machine gaps, not design.

Subagent system: the weakest part. Text is parsed from model output, coordination state is in memory, progress comes from a polled file the worker can write, and six-plus runtime code paths live inside the 5.5k-line OrchestratorService. It works on the happy path but breaks on restart and on clean-but-silent exits, and gives a worker's report the same authority as a user message.

4.1 Today's turn and subagent flow

1 Parent LLM writes a `mousse-actions` JSON block in plain text

▼ `parseActions` (LlmClient.ts:2650); invalid JSON silently skipped

2 `spawnAgentsOwned` (~300-line if-chain per agent kind)

reserve Agent + Task records in the parent's `agents.json` / `tasks.json`; create sparse "blast radius" worktree at `~/mousse/repositories/<id>/worktrees/agents/<id>`; start native loop, headless CLI, or PTY CLI (2 s sleep, then typed macro)

▼

3 Worker writes `.mousse/task-progress.json`; `TaskProgressMonitor` polls every 500 ms

▼

4 Batch settles; report injected as hidden role:'user' wake (100 ms debounce) **C1, C4**

▼

5 Parent emits `complete_task`, then `integrateAgentResult`

`snapshotOwnedParent` (auto-commit dirty parent), then `ChildAgentIntegrationService`: `git merge --no-ff <workerHead>` into the parent's thread worktree (journalled) **C3**

▼

6 Turn action receipt folds in child contributions, so undo reverts the child too

▼

7 Publish = explicit merge `--no-ff` in the user's primary checkout

A subagent is not a thread; it is records attached to the parent thread, with five overlapping nouns: Agent, Task, Episode, Mousse GUI agent, delegation batch.

4.2 Critical findings

C1 Daemon restart leaves the parent waiting forever

CRITICAL

Verified

What's wrong. delegationBatches, wakeQueues, wakeTimers and agentOwners are in memory only. restoreOnStartup marks agents interrupted but never rebuilds batches or wakes the parent. Headless agents are covered by neither restore condition, so they stay "running" while the task flips to interrupted. A wake is lost if the crash falls inside the 100 ms debounce. A second restore function, restoreAgentProgress, is only called by tests.

Why it matters. A restart silently strands delegated work; tests validate a path production never runs.

Fix direction. Persist batches, ownership and wakes (outbox); reconcile on startup; one SubagentSupervisor.

```
OrchestratorService.ts:623-629 (in-memory maps)
ThreadRuntimeManager.ts:284-308 (restoreOnStartup)
OrchestratorService.ts:3678 (restoreAgentProgress, test-only)
```

C2 Clean exit with no report hangs the agent forever

CRITICAL

Verified

What's wrong. The headless exit handler only acts on non-zero exit. There is no idle or wall-clock timeout. The "done" signal is a file the worker can forge or delete, and TaskProgressMonitor swallows parse errors.

Why it matters. A worker that exits cleanly without reporting is never noticed.

Fix direction. Process exit ends a run; exit 0 without a result becomes needs_review; add an idle watchdog.

```
OrchestratorService.ts:890-902
```

C3 Retrying after a merge conflict can commit the conflict markers

CRITICAL

Verified

What's wrong. A child conflict leaves the parent worktree mid-merge with recovery_required and no merge --abort. The 'conflict' status still passes shouldFinalizeAgent, so complete_task can be retried. The retry calls snapshotOwnedParent, then checkpointExistingTurn, which never calls assertNoPendingOperation or checks MERGE_HEAD. git add -A marks conflicts resolved and git commit --no-verify commits the markers.

Why it matters. Broken code with conflict markers lands in history.

Fix direction. Refuse checkpoints while MERGE_HEAD, REVERT_HEAD or unmerged paths exist; on conflict, auto-abort keeping the worker branch, or use an explicit "resolving" state.

```
ThreadActionService.ts:118-160, 300-303
```

C4 Worker reports injected into the parent as user instructions

CRITICAL

Verified

Detail unverified

What's wrong. checkDelegationBatches pastes the worker-written summary verbatim into a hidden role:user message that also says "emit complete_task with merge true". There is no delimiter, author tag or escaping.

Why it matters. A prompt-injection boundary for future room bots: worker text can steer the parent.

Fix direction. Wrap reports in a delimited, author-tagged block and remove the "merge true" instruction; long term, deliver reports as authored events.

```
checkDelegationBatches (OrchestratorService)
```

4.3 Major findings: subagents

Finding	Detail
MAJOR Commands are parsed text	Invalid JSON dropped silently; spawn_agents validation only checks it is an array; no idempotency key (a re-emitted block spawns again); native subagents re-parse actions (MousseAgentService.ts:1214).
MAJOR No limits	No concurrent-agent cap, no depth or fan-out counter, no token/cost/time budget. Recursion is prevented only by prompt and runtime ignore.
MAJOR No cascade on delete/stop	markThreadDeleted aborts only the parent turn; agents, PTYs, headless processes, pollers and maps survive.
MAJOR Races	Check-then-act on agent status across awaits; fire-and-forget validateAndMarkAgentReady.
MAJOR Many runtime paths	Six-plus runtime paths; PTY start relies on a 2 s sleep plus keystroke typing.

4.4 Major findings: worktrees and git

Finding	Detail
MAJOR Shared "current repo" singleton Verified	One WorktreeManager per profile with repoRoot = ... ?? process.cwd() (MmsProfileServices.ts:170); setRepoRoot swaps it on thread change. Mutation paths pass explicit paths (safe), but scanOrphanWorktrees, isValidatedAgentWorktreePath and getRepoRoot fallbacks still read it. init() creates <repoRoot>/.mousse-worktrees in whatever repo the daemon started in.
MAJOR Orphan scan looks in the wrong place Verified	It scans legacy in-repo .mousse-worktrees while agent worktrees live under ~/.mousse/repositories/<id>/worktrees/agents. Orphaned worktrees, mousse/agent/* branches and refs/mousse/agents/* accumulate forever (GC protects them).
MAJOR Agents share the user's node_modules Verified	linkSharedDependencies junctions each selective worktree's node_modules to the primary checkout. An agent's npm install or rm -rf changes the user's real dependencies; parallel agents share one folder.
MAJOR No dirty check at provision Verified	Thread worktrees branch from primary HEAD and ignore uncommitted changes (the collaboration plan claims otherwise).
MAJOR User repo config changed Verified	git config extensions.worktreeConfig true (WorktreeManager.ts:182).
MAJOR Publish in the live checkout	Merges into the human's checkout; a conflict leaves their tree mid-merge and blocks the thread; cannot serve several people publishing to one repo.
MAJOR Crash mid-git sticks a thread	Recovery calls requireClean and throws, never aborting a half-finished revert/merge; every turn start fails.
MAJOR Undo then retry falsely reports success Unverified	Repeated complete_task hits the idempotency short-circuit and says "Merged" though nothing applied.

Finding	Detail
MAJOR Hooks and environment	Repo hooks run unconfined during daemon git ops; inherited GIT_DIR / GIT_WORK_TREE / GIT_INDEX_FILE pass through; threadId/branchId are not validated before ref names.
MAJOR Shared workspace and non-git projects	Named agents with workspace:'shared' edit the parent's worktree directly; non-git projects fall back to the primary checkout with no isolation.

4.5 Minor findings and code quality

- **MINOR** Verified Dead code: WorktreeManager.mergeAndRemove and prepareForReady have no production callers. mergeAndRemove merges into the primary checkout with no lease or journal and commits a leftover MERGE_HEAD; delete it.
- **MINOR** Five duplicated git helpers (actions/git.ts, ThreadWorkspaceManager, WorkspaceGcService, WorkflowWorkspace, RepositoryIdentity) plus simple-git and GitService; revert/compensation builder copied 3 times; conflict handling copied 4 times.
- **MINOR** createWorktree has no rollback; Windows EBUSY during rollback is swallowed.
- **MINOR** resolveRepositoryIdentity spawns 3 to 4 git processes per call; journal and receipt list() are full scans ($O(n^2)$ over time); RepositoryLease has the same reclaim race, and a blocking git call over 30 s lets another process steal it.
- **MINOR** Test gaps: restart with live agents, wake loss, thread delete with live agents, snapshot mid-merge, crash mid-revert, retry after undo.

4.6 Strengths to keep

- **GitOperationCoordinator:** lock discipline; no deadlock path found.
- **ChildAgentIntegrationService:** pinned SHAs, idempotent operation IDs, refuses other repos, always merges into the owned thread worktree. complete_task cannot publish to the primary branch today.
- **Undo** via first-parent `-m 1 revert` (undoing a parent turn undoes merged children); undo after publish is blocked; outside commits are detected.
- Retention and ref release are journaled and CAS-based.
- Repository identity from git common dir; worktrees outside the user's repo; GC dry-run then purge, never forced.
- TaskWriterAuthority, the agent status transition table, and reserving records before discovery.

4.7 Recommended re-architecture

1. **A subagent is a child thread with a participant identity** (same stream/event-log primitive): own transcript, parentStreamId link, author Participant{kind:'agent', definitionId}. Reports reach the parent as authored events (fixes C4); room bots and subagents become the same thing.
2. **SubagentSupervisor outside OrchestratorService** with durable state: batches, ownership and wakes persisted (batchId on each run, wakes in an outbox). Startup reconciliation: live runs reattach; dead ones become interrupted plus a durable wake (fixes C1). One disposeThread() cascades cancellation.
3. **Native tool calls** (spawn_agent, complete_agent, message_agent) with JSON-schema validation, tool-result errors and idempotency keys; the supervisor enforces a concurrency cap, depth limit and per-run token/time/cost budgets.
4. **One AgentRuntime interface** (native LLM, headless CLI, SDK): process exit ends a run (exit 0 without result becomes needs_review, fixes C2); idle watchdog; retire PTY keystroke typing; progress as runtime events.

5. **A small git-state machine:** refuse checkpoint while MERGE_HEAD, REVERT_HEAD or unmerged paths exist (fixes C3); on conflict auto-abort keeping the worker branch, or an explicit "resolving" state; recovery from "prepared" aborts the half-finished op; stateless WorktreeManager (delete setRepoRoot, cwd fallback, legacy .mousse-worktrees); scan the real worktree base; per-worktree node_modules.
6. **Publish through a Mousse-owned integration worktree,** then update the target ref; leave the user's checkout alone or fast-forward only when clean. Pair with per-person undo ownership (LIFO undo on shared history will not work in rooms).

5. Roadmap

Phased plan

Phase	Goal	Checklist
1. Now days	Stop the bleeding	☐ Fix S1 to S5 and S9 ☐ Per-thread admission mutex (F4) ☐ Fix event cursor bug ☐ Delete dead UI copies ☐ ESLint (no-floating-promises) + Windows CI job
2. Kernel	Identity and streams	☐ Principal ☐ Stream + event log in SQLite (WAL) ☐ Per-stream executor ☐ Migrate threads onto streams; subagents as child threads
3. Authorization + SDK	One choke point, one client	☐ authorize() across handlers, dispatcher, channels, tools ☐ MousseClient SDK ☐ Move handlers.ts to DomainHandlerRegistry ☐ Split OrchestratorService; SubagentSupervisor
4. Mousse Bridge	Remote devices	☐ Transport under unified protocol ☐ Principal-bound device credentials ☐ Cursor resume ☐ Noise checked against external vectors
5. Rooms + bots	Product work	☐ Rooms and bots on the kernel ☐ Channels as adapters with durable inbox/outbox

Quick fixes (independent of the redesign)

- ☐ Add a MERGE_HEAD / REVERT_HEAD / unmerged-paths guard and assertNoPendingOperation to checkpointExistingTurn (C3).
- ☐ Treat headless exit 0 without completion as needs_review; include headless agents in restoreOnStartup (C2, part of C1).
- ☐ Delete restoreAgentProgress, mergeAndRemove, prepareForReady and their tests; test the real restore path.
- ☐ Point scanOrphanWorktrees at ~/.mousse/repositories/*/worktrees/agents.
- ☐ Wrap worker reports in a delimited, author-tagged block; remove the "merge true" instruction from wake text (C4).
- ☐ Add a per-thread concurrent-agent cap and a per-agent wall-clock timeout.
- ☐ Stop junctioning node_modules (or make it read-only and opt-in).